

UNIVERSIDAD DE GRANADA

Programación Web
Juan Manuel Fernández Luna

Práctica Evaluable II
Adaptación dinámica de la página web de la agencia de viaje de la práctica evaluable I

Diego Costas Moralejo
7 de junio de 2026

Introducción

La Práctica Evaluable 2 toma como punto de partida el sitio estático desarrollado en la PE1 y lo transforma en una aplicación web dinámica y completamente funcional. Si en la primera entrega toda la información estaba codificada directamente en el HTML, ahora la agencia dispone de un back-end real: una base de datos MySQL que almacena viajes, países, continentes y usuarios, y una capa de lógica en PHP orientado a objetos que actúa como puente entre esa base de datos y el navegador del usuario.

Las tres tecnologías principales que articulan esta práctica son PHP, MySQL y JavaScript, y cada una asume un papel diferenciado. PHP se encarga del procesamiento en el servidor: recibe peticiones, consulta la base de datos, construye los objetos del dominio y genera el HTML que finalmente llega al cliente. MySQL almacena de forma persistente toda la información de la agencia, con un esquema relacional que separa viajes, países y continentes en tablas distintas vinculadas mediante claves foráneas. JavaScript, por su parte, permanece completamente en el lado del cliente y cubre dos funciones concretas: la validación de formularios en tiempo real antes del envío y la gestión del carrusel de viajes en la portada.

Respecto a la práctica evaluable 1, los cambios más significativos son los siguientes. El catálogo de viajes, que antes era una lista estática de artículos HTML, ahora se genera automáticamente a partir de las filas de la base de datos, con paginación real controlada por parámetro GET. Se ha incorporado un sistema de autenticación mediante sesiones de PHP, con roles diferenciados: un usuario normal puede consultar el catálogo y buscar viajes, mientras que un administrador tiene acceso a un panel de gestión para crear, editar y eliminar destinos. El buscador de la portada, que en la PE1 no hacía nada, ahora lanza una consulta SQL real y muestra resultados filtrados por nombre de destino o por rango de fechas. Por último, el HTML se ha restructurado para eliminar los élementos superfluos y sustituirlos por etiquetas semánticas como <article>, <section>, <aside>, <details> o <dialog>, algo que tenía pendiente de la primera entrega y que el profesor subrayó en su comentario.

Base de datos

La base de datos se llama celtatravel y fue creada en un servidor MySQL local y después trasladada al servidor. Su diseño sigue un modelo relacional clásico con cuatro tablas: continentes, países, viajes y usuarios. La decisión de separar el destino geográfico en continente, país y viaje responde a un requisito funcional concreto: la barra lateral de la página de viajes muestra los destinos agrupados jerárquicamente, de modo que el usuario puede filtrar por país haciendo clic en un enlace. Si los países y continentes estuviesen embebidos como texto en la tabla de viajes, ese filtrado sería mucho más complejo y propenso a errores por inconsistencias de escritura.

Tabla continentes:

Es la tabla más sencilla. Solo tiene dos columnas: id (INT AUTO_INCREMENT PRIMARY KEY) y nombre (VARCHAR). Su razón de existir es servir como categoría de agrupación para los países. Contiene valores como “Europa”, “América”, “África” o “Asia”.

Tabla países:

Tiene tres columnas: id, nombre y idContinente. Esta última es una clave foránea que referencia a continentes.id, lo que permite obtener todos los países de un continente con una única consulta filtrada por idContinente.

Tabla viajes:

Es la tabla central del proyecto. Sus columnas son id, nombre, descripcion, frase, precio, fechaInicio, fechaFin, imagen, idPais y activo. El campo frase almacena una descripción corta pensada para el carrusel y las tarjetas de la lista. El campo imagen guarda solo el nombre del archivo, no la ruta completa, de forma que las páginas PHP construyen la ruta relativa anteponiendo imagenes/ según sea necesario. El campo activo es un TINYINT(1) que funciona como interruptor: los viajes con activo = 0 no aparecen en el catálogo público, pero el administrador los ve y puede editarlos. Esta solución es preferible a borrar físicamente los registros, ya que permite recuperar un viaje sin necesidad de volver a introducir todos sus datos.

Tabla usuarios:

Contiene id, nombreUsuario, email, password, tipo y fechaRegistro. El campo password nunca almacena la contraseña en texto plano: PHP usa password_hash() con el algoritmo bcrypt antes de insertar el registro, y password_verify() para comprobarla en el login. El campo tipo puede valer “usuario” (valor por defecto) o “admin”, y es el que determina si se muestra o no el panel de administración.

PHP: Clases

Todo el acceso a la base de datos está encapsulado en clases PHP que heredan de una clase abstracta común. Esta arquitectura reproduce el patrón proporcionado por el profesor y tiene una ventaja práctica clara: si en el futuro cambia el gestor de base de datos o las credenciales de conexión, solo hay que modificar dos archivos.

configuracion_inc.php:

Es el primer archivo que se incluye en la cadena de dependencias. Define seis constantes mediante define(): DB_DSN, que contiene la cadena de conexión PDO con el nombre del host y la base de datos; DB_USUARIO y DB_CONTRASENIA, con las credenciales de acceso; TAMANIO_PAGINA, que vale 9 y controla cuántos viajes se muestran por página en el catálogo paginado; y las cuatro constantes TABLA_USUARIOS, TABLA_VIAJES, TABLA_PAISES y TABLA_CONTINENTES, que contienen los nombres de las tablas. El uso de constantes evita que esos nombres queden dispersos por el código y hace que un cambio de nombre de tabla afecte a un único punto.

datosObject_class_inc.php:

Define la clase abstracta DataObject de la que heredan todas las demás. Su estructura tiene tres elementos fundamentales.

El primero es el array protegido $datos, que cada subclase redefine con los campos que le corresponden. El constructor recorre el array asociativo recibido (normalmente una fila de la base de datos) y copia en $datos solo las claves que ya existen en él, ignorando el resto. Esto garantiza que un objeto nunca tendrá campos inesperados aunque la consulta SQL devuelva columnas adicionales.

El segundo es el método público devolverValor($campo), que es la única forma de acceder a los datos desde fuera de la clase. Si el campo solicitado no existe en $datos, el método termina la ejecución con un mensaje de error, lo que facilita detectar errores de programación durante el desarrollo.

El tercero son los métodos estáticos protegidos conectar() y desconectar(). El primero crea y devuelve un objeto PDO con conexión persistente y modo de error por excepción, usando las constantes definidas en configuracion_inc.php. El segundo simplemente asigna null a la variable de conexión, lo que en PDO equivale a cerrar la conexión. Las subclases los invocan como parent::conectar() y parent::desconectar() al principio y al final de cada método de acceso a datos.

viaje_class_inc.php:

La clase Viaje es la más completa del proyecto porque gestiona todas las operaciones CRUD sobre la tabla viajes, además de las consultas de listado, búsqueda y filtrado.

El método obtenerViajes($filaInicio, $numeroFilas, $soloActivos = false) es el que usa la paginación. Acepta un parámetro opcional booleano que, si vale true, añade la cláusula WHERE activo = 1 a la consulta. Esto permite reutilizar el mismo método tanto para el catálogo público (solo viajes activos) como para el panel del administrador (todos los viajes). La consulta usa SQL_CALC_FOUND_ROWS junto con SELECT FOUND_ROWS() en una segunda consulta para obtener el total de filas sin ejecutar una segunda SELECT COUNT(*), y devuelve un array con dos elementos: la lista de objetos Viaje y el total de filas.

El método buscar($destino, $fechaInicio, $fechaFin) construye la consulta SQL de forma condicional según los parámetros que estén rellenos. Si solo hay destino, filtra por nombre con LIKE. Si solo hay fechas, filtra por rango. Si hay ambos, combina las dos condiciones. Si no hay nada, devuelve false directamente sin llegar a abrir conexión. El criterio de fechas busca viajes cuya fechaInicio sea anterior o igual a la fecha solicitada y cuya fechaFin sea posterior o igual, es decir, viajes que estén en curso durante el período buscado.

Los métodos insertar() y modificar() reciben todos los campos del viaje como parámetros individuales y utilizan sentencias preparadas con bindValue() para cada uno de

ellos, evitando cualquier posibilidad de inyección SQL. El método eliminar() funciona de forma similar pero solo necesita el id. Los tres devuelven true si la operación tiene éxito y false si se produce una excepción PDO.

El método obtenerViajesPorPais($idPais) filtra los viajes activos de un país concreto y se usa en dos lugares: en viajes.php para mostrar el catálogo filtrado cuando el usuario hace clic en un país del menú lateral, y en viaje_detalle.php para mostrar la sección de "Viajes relacionado".

usuario_class_inc.php:

La clase Usuario gestiona el registro y la autenticación. El método obtenerPorUsuario($nombreUsuario) busca un usuario por su nombre de usuario y devuelve un objeto Usuario o null. En la página de inicio, ese objeto se usa inmediatamente después para llamar a verificarPassword($password), que internamente invoca password_verify() comparando la contraseña introducida con el hash almacenado en la base de datos.

El método registrar() hace la inserción con password_hash() aplicado a la contraseña antes de bindValue(), de modo que el hash llega a la base de datos, nunca la contraseña en claro. Existe también registrarAdmin(), que inserta el valor 'admin' en el campo tipo, pensado para crear el usuario administrador directamente desde código sin necesidad de modificar la base de datos a mano.

pais_class_inc.php y continente_class_inc.php:

Estas dos clases son más simples. Continente solo tiene obtenerTodos(), que devuelve todos los continentes ordenados por nombre. Pais tiene obtenerTodos() para el formulario de creación de viajes, donde el administrador necesita elegir el país de destino, y obtenerPorContinente($idContinente), que devuelve solo los países de un continente concreto y se usa en el menú lateral de viajes.php para construir el árbol desplegable de destinos.

PHP: Páginas

index.php:

La portada es la página más compleja del proyecto porque reúne varias funciones: actuar como pantalla de bienvenida, gestionar el login del usuario y albergar el buscador y el carrusel de viajes.

Al cargarse, lo primero que hace el código PHP es llamar a Viaje::obtenerTodos() para recuperar todos los viajes activos. Esta lista se necesita para el carrusel, que se hace con JavaScript, y la forma de pasarle los datos es mediante json_encode() dentro de un bloque <script> incrustado en el HTML. Así el JavaScript del carrusel tiene acceso a los datos sin necesidad de hacer peticiones AJAX.

La gestión del login se realiza mediante un formulario POST situado en cabecera.php que apunta a index.php. Es esta página la que procesa la petición: recupera el nombre de usuario y la contraseña enviados, busca el usuario en la base de datos y verifica la contraseña. Si la verificación tiene éxito, guarda el nombre de usuario y el tipo de rol en la sesión y redirige mediante header('Location: index.php') para evitar el reenvío del formulario al recargar. Si falla, asigna un mensaje de error a la variable $error, que la cabecera muestra en rojo junto al formulario de login.

El buscador es un formulario GET con tres campos: destino, fecha_inicio y fecha_fin. Al enviarse, redirige a viajes_buscados.php con esos valores como parámetros en la URL. La validación previa al envío la realiza JavaScript, no PHP: el script validacion.js comprueba que no se envíe el formulario completamente vacío y que las fechas, si se introducen, sean coherentes entre sí. Además, en la portada los campos de fecha están inicializados con Flatpickr, que reemplaza el selector nativo del navegador por uno más visual y traducido al español.

El carrusel se construye sobre un <article> dentro de un <a> que funciona como enlace al viaje mostrado en cada momento. Las flechas de navegación son elementos a los que JavaScript añade los listeners de clic.

viajes.php (Catálogo de viajes):

Esta página muestra el catálogo paginado de viajes activos y el menú lateral para filtrar por país.

Al iniciarse, calcula la página actual a partir del parámetro GET 'pagina' (si no llega, se asume 1) y a partir de ella calcula filaInicio multiplicando por la constante TAMANIO_PAGINA. Llama a Viaje::obtenerViajes() con el flag soloActivos = true, lo que le devuelve tanto el array de viajes como el total de filas, necesario para calcular cuántas páginas hay en total con ceil().

Al hacer clic en un país del menú lateral, la URL incluye el parámetro idPais, y la página detecta ese parámetro y llama a Viaje::obtenerViajesPorPais() en lugar de usar la lista paginada. En ese caso, la paginación no se muestra, porque se presentan todos los viajes del país de una vez.

Las tarjetas de viaje se renderizan con un bucle foreach sobre el array de viajes. A cada tarjeta se le asigna una de tres clases CSS (viaje1, viaje2 o viaje3) según el índice módulo 3, lo que produce tres disposiciones visuales distintas que alternan cíclicamente y dan variedad visual al listado. Cada tarjeta es un <article> envuelto en un <a> que apunta a viaje_detalle.php?id=X (la página con los detalles del viaje con id X).

viaje_detalle.php (Página que detalla más en profundidad cada viaje):

Esta página recibe el parámetro GET 'id' y muestra toda la información de un viaje concreto. Si el parámetro no llega, o si el id no corresponde a ningún viaje en la base de datos, el código redirige inmediatamente a viajes.php con header().

Una vez recuperado el viaje, busca los viajes relacionados llamando a Viaje::obtenerViajesPorPais() con el idPais del viaje actual, y después filtra con array_filter() para excluir el propio viaje de esa lista. Los viajes relacionados se muestran en un <aside> con enlaces a sus respectivas fichas.

viajes_buscados.php (Página que muestra los resultados del formulario de búsqueda de index.php):

Esta página es sencilla en su estructura PHP: recoge los tres parámetros GET posibles (destino, fecha_inicio y fecha_fin), los pasa a Viaje::buscar() y muestra los resultados. Si el array devuelto está vacío, muestra un mensaje indicando que no se han encontrado viajes. Si hay resultados, los renderiza con el mismo patrón de tarjetas de viaje que en viajes.php, con la misma rotación de clases CSS y los mismos enlaces a la ficha de cada viaje.

altausuarios.php (Registro de usuarios):

Tiene cuatro fieldset: datos personales (nombre, apellidos, DNI, fecha de nacimiento, teléfono y correo), datos de acceso (nombre de usuario y contraseña), preferencias de viaje (destino favorito con datalist, tipo de viaje con select, opción de recibir ofertas con radio y servicios de interés con checkboxes) y un campo de comentarios libre.

Cuando se envía el formulario, el código PHP recoge solo los campos necesarios para la base de datos (nombreUsuario, email y password) y llama a Usuario::registrar(). Si el nombre de usuario o el email ya existen, la base de datos devolverá un error de clave duplicada que PDO convierte en una excepción, el método devuelve false y la página muestra un mensaje de error. Si el registro es correcto, redirige a altacorrecta.php.

Los demás campos del formulario (apellidos, DNI, teléfono, etc.) se validan en el lado del cliente mediante JavaScript pero no se almacenan en la base de datos, ya que el esquema actual de la tabla usuarios no los contempla. Esto es coherente con el enfoque de la práctica, que prioriza la lógica de negocio sobre la exhaustividad del modelo de datos.

El campo de fecha de nacimiento usa Flatpickr configurado con la opción maxDate: 'today', de modo que el selector no permite elegir fechas futuras.

admin/gestionar_viajes.php (Panel de administración):

Esta página es el punto de entrada del área de administración. Lo primero que hace al cargarse es comprobar que la sesión tiene un usuario autenticado con rol admin. Si no es así, redirige a index.php inmediatamente. Esta comprobación se repite en todas las páginas del

directorio admin/, por lo que no es posible acceder a ellas directamente por URL sin estar autenticado como administrador.

La página también gestiona la acción de eliminar: si llega el parámetro GET 'accion=eliminar' junto con un id, llama a Viaje::eliminar() y redirige de vuelta a la misma página. En el HTML, el botón de eliminar no es un enlace directo a esa URL, sino que abre primero un diálogo de confirmación.

El diálogo de confirmación usa el elemento HTML nativo <dialog>. Cuando el usuario hace clic en cualquier botón “Eliminar” de las tarjetas, JavaScript intercepta el clic, almacena la URL de eliminación en el atributo data-url del diálogo y lo abre con showModal(). Si el usuario confirma, window.location.href navega a esa URL; si cancela, el diálogo se cierra con modal.close(). De esta forma se evita el borrado accidental de viajes.

El panel lateral tiene un enlace a formulario_viaje.php sin parámetros para crear un nuevo viaje. Cada tarjeta tiene también un enlace de edición que apunta a formulario_viaje.php?id=X donde aparece el mismo formulario pero ya con los campos rellenados tal y cómo están en ese momento.

admin/formulario_viaje.php (Crear y editar viajes):

Este formulario funciona en dos modos. Si llega el parámetro GET 'id', está en modo edición: recupera el viaje correspondiente con Viaje::obtenerViaje() y rellena previamente todos los campos del formulario con sus valores actuales mediante expresiones PHP en los atributos value. Si no llega ninguna id, está en modo creación y los campos aparecen vacíos. El título de la página y el texto del botón de envío cambian en consecuencia mediante el operador ternario de PHP.

Al recibir un POST, el código determina si debe llamar a Viaje::modificar() o a Viaje::insertar() según el modo activo. El checkbox de «Viaje activo» se trata de forma especial: si está marcado, el valor que llega en $_POST es el string 'on', y mediante isset($_POST['activo']) ? 1 : 0 se convierte al entero correspondiente.

El selector de país es un <select> cuyos <option> se generan dinámicamente con Pais::obtenerTodos(). En modo edición, el <option> correspondiente al país actual del viaje tiene el atributo 'selected' añadido también por PHP.

sugerencias.php:

El formulario de esta página está validado por JavaScript pero su envío no persiste en la base de datos, ya que no hay tabla de sugerencias en el esquema actual.

includes/cabecera.php:

La cabecera se incluye en todas las páginas con require_once. Comprueba si la variable de sesión 'usuario' está definida. Si lo está, muestra el nombre y el rol del usuario conectado, un enlace al panel de administración (solo para admins) y el icono de logout. Si no lo está, muestra el formulario de login y el enlace de registro.

La navegación usa la variable $paginaActual, que cada página define antes de incluir la cabecera (por ejemplo, <?php $paginaActual = 'viajes'; ?>). En la cabecera, cada enlace del menú compara su nombre con $paginaActual y, si coincide, añade la clase CSS 'nav-activo', que aplica un estilo visual diferente para indicar al usuario en qué página se encuentra.

JavaScript

validacion.js:

El archivo validacion.js centraliza la validación del lado del cliente para los cinco formularios del sitio. Funciona con un patrón común: al principio busca cada formulario por su id, y si existe en la página actual añade un listener al evento 'submit'. Dentro del listener, una variable booleana 'valido' empieza en true y se pone a false si alguna comprobación falla. Si al final vale false, se llama a evento.preventDefault() para cancelar el envío.

Los mensajes de error se muestran mediante la función mostrarError(campo, mensaje), que busca si el elemento siguiente al campo en el DOM es un con la clase 'error-validacion'. Si no existe, lo crea e inserta justo después del campo con insertBefore(). Si ya existe de una validación anterior, reutiliza el mismo y actualiza su contenido. limpiarError() simplemente llama a mostrarError() con un mensaje vacío, lo que hace que el span quede visible pero sin texto.

El formulario de alta valida nombre, apellidos, DNI mediante la expresión regular /^[0-9]{8}[A-Za-z]$/, fecha de nacimiento con el formato AAAA-MM-DD, teléfono con /^[0-9]{9}$/, correo con /^[^\s@]+@[^\s@]+\.[^\s@]+$/, nombre de usuario, contraseña con mínimo de seis caracteres, destino favorito, tipo de viaje, si quiere recibir ofertas y presupuesto numérico entre 300 y 5000.

La validación del buscador tiene una lógica propia. Primero comprueba de forma independiente si los tres campos están vacíos a la vez, en cuyo caso pide al menos un criterio de búsqueda. Después, con bloques else if independientes del anterior, comprueba si solo se ha introducido una de las dos fechas y si la fecha de fin es anterior a la de inicio. Esta separación en dos niveles de validación es intencional: el primer bloque comprueba que haya algo que buscar; el segundo comprueba que las fechas, si las hay, sean coherentes.

El formulario de gestión de viajes valida nombre, frase promocional, descripción, precio numérico positivo, fechas en formato AAAA-MM-DD con coherencia entre ellas, y nombre de imagen.

carrusel.js:

El carrusel de la portada se basa en el array JavaScript 'viajes' que PHP inyecta en el HTML mediante json_encode(). Cada elemento del array tiene id, nombre, frase, imagen y url. El archivo carrusel.js no necesita saber de dónde vienen esos datos: los consume directamente del array global.

La variable indiceActual empieza en 0. La función mostrarViaje(indice) actualiza cuatro elementos del DOM: el src y el alt de la imagen, el texto del nombre y el texto de la frase. También actualiza el href del enlace que envuelve toda la tarjeta, de modo que al hacer clic en cualquier parte del carrusel se navega al detalle del viaje que se está mostrando.

El evento DOMContentLoaded inicializa el carrusel con el primer viaje. Las flechas de navegación tienen listeners de clic que incrementan o decrementan indiceActual y, en ambos casos, aplican el operador módulo sobre la longitud del array. Esto hace que el carrusel sea cíclico: al llegar al último viaje y pulsar la flecha derecha se vuelve al primero, y al estar en el primero y pulsar la izquierda se salta al último.

Innovaciones

Flatpickr como selector de fechas:

En lugar del selector de fechas nativo del navegador, que tiene apariencias muy diferentes en Chrome, Firefox y Safari, se ha integrado la librería Flatpickr en los formularios de alta de usuarios (campo de fecha de nacimiento), en el buscador de la portada (campos de fecha de inicio y fin) y en el formulario de gestión de viajes del administrador. Flatpickr se inicializa con la localización española y el formato de fecha AAAA-MM-DD, que es el que espera MySQL. La integración se realiza mediante CDN, añadiendo el CSS y el JavaScript de la librería en el <head> de las páginas que lo necesitan, y llamando a flatpickr("#id_campo", opciones) al final del body.

Diálogo nativo de confirmación con <dialog>:

La acción de eliminar un viaje en el panel de administración está protegida por un diálogo de confirmación implementado con el elemento HTML5 <dialog>, que es la alternativa moderna a window.confirm() y a los modales basados en librerías externas. El diálogo contiene solo dos botones: confirmar y cancelar. JavaScript almacena en el dataset del diálogo la URL de eliminación correspondiente a cada viaje y la usa si el usuario confirma. Esta solución es completamente estándar y no requiere ninguna dependencia externa.

Resaltado de la página activa en el menú:

El menú de navegación de la cabecera distingue visualmente la página que el usuario tiene abierta en cada momento. Para lograrlo sin consultar la URL del navegador desde JavaScript, cada página PHP define la variable $paginaActual antes de incluir cabecera.php. La cabecera compara esa variable con un identificador fijo por cada enlace del menú y, si coinciden, añade la clase CSS 'nav-activo' al elemento <a>. En el CSS, esa clase aplica un color diferente y un subrayado que señala visualmente la sección activa.

Uso de IA

Durante el desarrollo de esta práctica se ha utilizado Claude (Anthropic) como herramienta de apoyo puntual para la depuración de errores concretos que han surgido durante el desarrollo. También se ha consultado el funcionamiento de librerías y elementos externos incorporados como innovación, concretamente la configuración del módulo de localización de Flatpickr y el comportamiento del elemento <dialog> nativo de HTML5. Adicionalmente, se han resuelto dudas puntuales sobre cómo pasar datos de PHP a JavaScript mediante json_encode() para alimentar el carrusel de la portada, sobre el uso de SQL_CALC_FOUND_ROWS junto con FOUND_ROWS() para la paginación, sobre el uso correcto de bindValue() con los tipos de dato PDO en las sentencias preparadas, y sobre el funcionamiento de los closures en PHP y el uso de use para capturar variables del ámbito exterior, necesario para filtrar los viajes relacionados en la ficha de detalle.